Functional Programming with Go

Concepts of Programming Languages
7 November 2019

Johannes Weigend
Rosenheim Technical University

What is Functional Programming?



-
-
-

Functional Programming – Characteristics

-
-
-
-

Functional programming languages are categorized into two groups

- Pure
- Impure

Functional programming offers the following advantages

-
-

run concurrently

-

lazy evaluation

-

Functions are Values

```
func aBlock(i int) {
    fmt.Printf("Entering block: i=%v\n", i)
}

func do(f func (int), loops int) {
    for i := 0; i < loops; i++ {
        f(i)
    }
}

func main() {
    do(aBlock, 5)
}
```

(https://play.golang.org/p/3Lg5BCPiiPA)

Many Functional Languages only support Single Argument Functions

-

```
// ADD with 2 parameters
ADD := func(x, y int) int {
    return x + y
}
```

```
ADD(1,2) -> 3
```

```
// Curried ADD
ADDC := func(x int) func(int) int {
    return func(y int) int {
        return x + y
    }
}
```

```
ADDC(1)(2) -> 3
```

Functional Composition

```
// Function f()
f := func(x int) int {
    return x * x
}

// Function g()
g := func(x int) int {
    return x + 1
}

// Functional Composition: (g◦f)(x)
gf := func(x int) int {
    return g(f(x))
}

fmt.Printf("%v\n", gf(2)) // --> 5
```

Functional Composition (2)

```
// Type any makes the code readable
type any interface{}
type function func(any) any
```

```
compose := func(g, f function) function {
    return func(x any) any {
        return g(f(x))
    }
}
```

```
square := func(x any) any { return x.(int) * x.(int) }

fmt.Printf("%v\n", compose(square, square)(2)) // --> 4*4 = 16

fmt.Printf("%v\n", compose(compose(square, square), square)(2)) // --> 256
```

Closures (Only impure if you modify the closed-over variable)

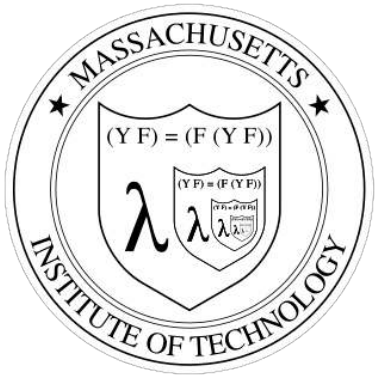
```
// intSeq returns another function, which we define anonymously in the body of intSeq.
// The returned function closes over the variable i to form a closure.
func intSeq() func() int {
    i := 0
    return func() int {
        i++
        return i
    }
}
func main() {
    // We call intSeq, assigning the result (a function) to nextInt.
    // This function value captures its own i value, which will be updated each time we call nextInt.

    nextInt := intSeq()
    // See the effect of the closure by calling nextInt a few times.
    fmt.Println(nextInt())
    fmt.Println(nextInt())

    // To confirm that the state is unique to that particular function, create and test a new one.
    newInts := intSeq()
    fmt.Println(newInts())
}
```

Run

History: The Lambda Calculus



-
-
-

(https://www.youtube.com/watch?v=eis11j_iGMs)

Summary of the Introduction to Lambda Calculus

-
-
-
-
-
-

Lambda Calculus in Go

(https://play.golang.org/p/1bLmezdD2zt)

```
// Lambda Calculus in Golang --> See Video Graham Hutton
// https://www.youtube.com/watch?v=eis11j_iGMs

// This is the key: A Recursive function definition for all functions!!!
type fnf func(fnf) fnf

ID := func(x fnf) fnf { return x }

// TRUE as function: λx.λy.x
True := func(x fnf) fnf {
    return func(y fnf) fnf {
        return x
    }
}

// FALSE as function: λx.λy.y
False := func(x fnf) fnf {
    return func(y fnf) fnf {
        return y
    }
}
```

Application

```
fmt.Printf("Id = %p\n", ID)
fmt.Printf("True = %p\n", True)
fmt.Printf("False = %p\n", False)

// debugging functions
f := func(x fnf) fnf { fmt.Printf("f()\n"); return x }
g := func(y fnf) fnf { fmt.Printf("g()\n"); return y }

// select and call first function f(ID)
False(False)(True)(f)(g)(ID)

// select and call second function g(ID)
True(False)(True)(f)(g)(ID)
```

Lambda Calculus in Go: NOT

```
// NOT as function: λb.b false true
Not := func(b fnf) fnf {
    return b(False)(True)
}

// should print false
fmt.Printf("Not(True) = %p\n", Not(True))

// should print true
fmt.Printf("Not(False) = %p\n", Not(False))

// select and call first function f(ID)
Not(False)(f)(g)(ID)

// select and call second function g(ID)
Not(True)(f)(g)(ID)
```

Functional Numbers

```
// Functional Numbers 1
ONCE := func(f fnf) fnf {
    return func(x fnf) fnf {
        return f(x)
    }
}
```

```
// Functional Numbers 2
TWICE := func(f fnf) fnf {
    return func(x fnf) fnf {
        return f(f(x))
    }
}
```

```
// Function Numbers 3
THRICE := func(f fnf) fnf {
    return func(x fnf) fnf {
        return f(f(f(x)))
```

```
}
```

```
}
```

Functional Numbers

```
// Functional Numbers SUCCESSOR(N) = N + 1
SUCCESSOR := func(w fnf) fnf {
    return func(y fnf) fnf {
        return func(x fnf) fnf {
            return y(w)(y)(x)
        }
    }
}
```

```
Printer := func(x fnf) fnf { fmt.Print("."); return x }
```

```
SUCCESSOR(TWICE)(Printer)(ID)
fmt.Println("SUCCESSOR(TWICE) = 3")

SUCCESSOR(THRICE)(Printer)(ID)
fmt.Println("SUCCESSOR(THRICE) = 4")
```

Lambda Calculus in JavaScript

```
TRUE = a => b => a;
FALSE = a => b => b;
NOT = f => a => b => f(b)(a);

f = x => x + 10
g = x => x + 20

TRUE(f)(g)(3)       // -> 13
FALSE(f)(g)(3)      // -> 23

NOT(TRUE)(f)(g)(3)  // -> 23
NOT(FALSE)(f)(g)(3) // -> 13
```

(https://www.youtube.com/watch?v=3VQ382QG-y4)

Famous Functional Languages inspired by the Lamda Calculus

-

(https://www.youtube.com/watch?v=1jZ7j21g028)

-
-
-
-
-

Palindrome Problem in Functional (pure) Languages

-
-

Palindrome Problem in Functional (impure) Languages

-
-
-

Functions as First Class Citizens in Go

-
-
-
-
-

Sample from the Go Standard Library

-
-

Go Streams with Generics (Go 1.18+)

-

```
func TestMapFilterReduce(t *testing.T) {
    // Type-safe slice of strings (no more interface{}).
    stringSlice := []string{"a", "b", "c", "1", "D"}

    // Map/Reduce with type-safe generics
    result := Of(stringSlice).
        Map(toUpperCase).
        Filter(notDigit).
        Reduce(concat)

    if result != "A,B,C,D" {
        t.Error(fmt.Sprintf("Result should be 'A,B,C,D' but is: %v", result))
    }
```

Classic Word Count Sample with ReduceTo

```
//
// ========================
// Classic wordcount sample
// ========================

func TestWordCount(t *testing.T) {
    words := []string{"a", "a", "b", "b", "D", "a"}

    // Using ReduceTo for type-transforming reduction (string -> map[string]int)
    wordCounts := ReduceTo(Of(words), make(map[string]int), func(counts map[string]int, word string) map[string]
        counts[word]++
        return counts
    })

    // Print results
    for k, v := range wordCounts {
        fmt.Printf("%v:%v, ", k, v)
    }
    fmt.Println()

    // Verify counts
    if wordCounts["a"] != 3 {
        t.Errorf("Expected 'a' count to be 3, got %d", wordCounts["a"])
    }
    if wordCounts["b"] != 2 {
        t.Errorf("Expected 'b' count to be 2, got %d", wordCounts["b"])
    }
    if wordCounts["D"] != 1 {
        t.Errorf("Expected 'D' count to be 1, got %d", wordCounts["D"])
```

```
    }
}
```

Questions

-
-

Summary

-
-
-
-
-

Thank you

Johannes Weigend
Rosenheim Technical University
johannes.weigend@weigend.de (mailto:johannes.weigend@weigend.de)
http://www.weigend.de (http://www.weigend.de)